Placement Tracker: Database Persistence, Robust AI & RBAC Implementation Guide

Standalone Implementation & Incremental Refactoring Guide for Antigravity
College Placement Drive and Student Application Tracker (SIH 2026)
Referenced from the SIH 2026 Internal Practical Assessment Specification

1. System Objective & Modernization Blueprint

This specification details how to refactor and upgrade the existing `placement-tracker/` folder to eliminate demo limitations and make the system production-grade:

- **True Data Persistence & Multi-User Collaboration**:
  - Transition from ephemeral browser cookie chunking (`pt_data_*`) to a real relational backend (Node.js/Express with Prisma ORM and SQLite/PostgreSQL).
  - Ensure real-time multi-client updates: when a Placement Officer updates an application status or creates a drive, changes are immediately reflected across student and admin interfaces.
- **Deterministic & Semantic AI Placement Assistant**:
  - Fix the input-matching bug observed in the student assistant where suggested prompts (e.g., "Where did I apply?") trigger fallback error responses.
  - Implement robust token normalization, fuzzy string matching (via Fuse.js), and extensible intent handlers.
- **Server-Side Role-Based Access Control (RBAC) & Security**:
  - Enforce authentication via JWT stored in secure HTTP-only cookies.
  - Restrict mutation endpoints (updating drives, application stages, and user accounts) strictly to authorized roles (`OFFICER`, `ADMIN`).
  - Store system activity audit logs immutably in the database rather than client cookies.

2. Directory Structure & File Modification Map

Execute modifications within the existing folder layout:t

```
placement-tracker/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma           # [NEW/UPDATE] Relational models for Users, Drives, Applications, Logs
│   │   └── seed.js                 # [NEW/UPDATE] Seeding script for demo records (~100 applications)
│   ├── src/
│   │   ├── middleware/auth.js      # [NEW] JWT & Role verification middleware
│   │   ├── routes/
│   │   │   ├── auth.routes.js      # [NEW] Login, logout, session verification
│   │   │   ├── drives.routes.js    # [NEW] Drive listings and creation
│   │   │   ├── applications.routes.js # [NEW] Student applications & stage mutation
│   │   │   └── audit.routes.js     # [NEW] Append-only audit logging
│   │   ├── server.js               # [UPDATE] Express application bootstrap with CORS & routes
│   │   └── db.js                   # [NEW] Prisma client singleton instance
│   ├── package.json                # [UPDATE] Add cors, express, jsonwebtoken, bcryptjs, @prisma/client
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── services/
│   │   │   └── api.js              # [NEW] Axios instance configured with baseURL and credentials
│   │   ├── context/
│   │   │   └── PlacementContext.jsx  # [NEW] Global state provider for drives, apps, and live sync
│   │   ├── lib/
│   │   │   └── assistantMatcher.js   # [NEW/UPDATE] Fixed fuzzy intent matching for AI Assistant
│   │   ├── components/
```

```
│   │   │   └── AIAssistantModal.jsx   # [UPDATE] Integrate assistantMatcher & context data
│   │   ├── pages/
│   │   │   ├── StudentDashboard.jsx   # [UPDATE] Consume live context applications & drives
│   │   │   ├── OfficerDashboard.jsx   # [UPDATE] Stage editing with optimistic UI updates
│   │   │   └── AdminDashboard.jsx     # [UPDATE] Live user management & audit logs
│   │   └── App.jsx                 # [UPDATE] Wrap with PlacementProvider
│   └── package.json                  # [UPDATE] Add axios, fuse.js
└── README.md                         # [UPDATE] Document new backend setup & environment variables
```

3. Backend Implementation (Express + Prisma ORM)

3.1 Prisma Schema (`backend/prisma/schema.prisma`)

```prisma
datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum Role {
```

```
  STUDENT
  OFFICER
  ADMIN
}

enum ApplicationStage {
  APPLIED
  UNDER_REVIEW
  SHORTLISTED
  ASSESSMENT
  INTERVIEW
  OFFERED
  REJECTED
}

enum OfferStatus {
  PENDING
  OFFERED
  NOT_OFFERED
  ACCEPTED
  DECLINED
```

```
}

model User {
  id           String       @id @default(cuid())
  email        String       @unique
  passwordHash String
  fullName     String
  rollNumber   String?      @unique
  department   String?
  cgpa         Float?
  activeBacklogs Int        @default(0)
  role         Role         @default(STUDENT)
  isActive     Boolean      @default(true)
  createdAt    DateTime     @default(now())
  updatedAt    DateTime     @updatedAt
  applications Application[]
  auditLogs    AuditLog[]
}

model Company {
```

```
  id          String          @id @default(cuid())
  name        String          @unique
  industry    String
  minPackage  Float?
  maxPackage  Float?
  websiteUrl  String?
  createdAt   DateTime        @default(now())
  drives      PlacementDrive[]
}

model PlacementDrive {
  id             String       @id @default(cuid())
  companyId      String
  company        Company      @relation(fields: [companyId], references: [id], onDelete: Cascade)
  roleTitle      String
  minCgpa        Float        @default(0.0)
  driveDate      DateTime
  status         String       @default("Upcoming") // Upcoming, Ongoing, Completed
  eligibleDepts  String       // Comma-separated: "CSE,IT,ECE"
```

```
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt
  applications    Application[]
}

model Application {
  id          String          @id @default(cuid())
  studentId   String
  student     User            @relation(fields: [studentId], references: [id], onDelete: Cascade)
  driveId     String
  drive       PlacementDrive  @relation(fields: [driveId], references: [id], onDelete: Cascade)
  stage       ApplicationStage @default(APPLIED)
  offerStatus OfferStatus     @default(PENDING)
  packageOffered Float?
  outcome     String          @default("Pending") // Placed, Not Placed, Pending
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt
  @@unique([studentId, driveId])
}
```

```
model AuditLog {
  id           String   @id @default(cuid())
  userId       String?
  user         User?    @relation(fields: [userId], references: [id], onDelete: SetNull)
  actorName    String
  role         String
  action       String
  impactedEntity String
  ipAddress    String?  @default("127.0.0.1")
  timestamp    DateTime @default(now())
}
```

3.2 Authentication & Authorization Middleware (`backend/src/middleware/auth.js`)

```
const jwt = require('jsonwebtoken');

const JWT_SECRET = process.env.JWT_SECRET || 'sih-2026-secure-secret-key';

|---|

function authenticateToken(req, res, next) {
  const token = req.cookies?.pt_session || req.headers.authorization?.split(' ')[1];
```

```
  if (!token) {
    return res.status(401).json({ error: 'Access denied. No authentication token provided.' });
  }
  try {
    const verified = jwt.verify(token, JWT_SECRET);
    req.user = verified;
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Invalid or expired session token.' });
  }
}

function requireRole(roles = []) {
  return (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden: Insufficient privileges.' });
    }
    next();
  };
```

```
}

module.exports = { authenticateToken, requireRole, JWT_SECRET };
```

3.3 Core Express Server (`backend/src/server.js`)

```
const express = require('express');

const cors = require('cors');

const cookieParser = require('cookie-parser');

const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();

const app = express();

app.use(cors({

  origin: process.env.FRONTEND_URL || 'http://localhost:5173',

  credentials: true,

}));

app.use(express.json());

app.use(cookieParser());

// Mount application routes

app.use('/api/auth', require('./routes/auth.routes'));

app.use('/api/drives', require('./routes/drives.routes'));

app.use('/api/applications', require('./routes/applications.routes'));
```

```
app.use('/api/audit', require('./routes/audit.routes'));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Placement Tracker backend listening on port ${PORT}`);
});
```

4. Frontend Integration Layer

4.1 Axios API Client (`frontend/src/services/api.js`)

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  withCredentials: true,
  headers: {
    'Content-Type': 'application/json',
  },
});

export default api;
```

4.2 Robust AI Assistant Fuzzy Matcher (`frontend/src/lib/assistantMatcher.js`)

This module replaces fragile string comparisons with regex normalization and Fuse.js fuzzy scoring:import Fuse from 'fuse.js';

```
const INTENTS = [
  {
    id: 'MY_APPLICATIONS',
    patterns: [
      'where did i apply',
      'show my applications',
      'what companies did i apply to',
      'list my applied companies',
      'my applications',
      'where have i applied'
    ]
  },
  {
    id: 'SHORTLISTED_STATUS',
    patterns: [
      'was i shortlisted',
```

```
    'am i shortlisted anywhere',
    'show my shortlisted companies',
    'shortlist status',
    'shortlisted drives'
  ]
},
{
  id: 'OFFERS_RECEIVED',
  patterns: [
    'did i get an offer',
    'did i receive any offers',
    'have i received an offer',
    'what is my package',
    'show my offers',
    'offer letters'
  ]
},
{
```

```
    id: 'PENDING_STATUS',
    patterns: [
      'show my application status',
      'which applications are pending',
      'pending drives',
      'am i in interview round'
    ]
  },
  {
    id: 'CAMPUS_SUMMARY',
    patterns: [
      'how many companies visited campus',
      'total companies visited',
      'campus placement drives'
    ]
  }
];

const flattenedPatterns = INTENTS.flatMap(intent =>
  intent.patterns.map(pattern => ({
```

```
    intentId: intent.id,
    pattern
  }))
);
const fuse = new Fuse(flattenedPatterns, {
  keys: ['pattern'],
  threshold: 0.45,
  ignoreLocation: true
});
export function matchAssistantQuery(userInput) {
  if (!userInput || typeof userInput !== 'string') return null;
  const normalized = userInput
    .toLowerCase()
    .replace(/[?!.,]/g, '')
    .trim();
  // 1. Direct Normalized Check
  for (const intent of INTENTS) {
    if (intent.patterns.some(p => p.toLowerCase().replace(/[?!.,]/g, '').trim() ===
normalized)) {
```

```
      return intent.id;
    }
  }
  // 2. Fuzzy Match via Fuse.js
  const results = fuse.search(normalized);
  if (results.length > 0) {
    return results[0].item.intentId;
  }
  return null;
}
```

4.3 Global State & Real-Time Sync (`frontend/src/context/PlacementContext.jsx`)

```
import React, { createContext, useContext, useState, useEffect, useCallback } from 'react';
import api from '../services/api';
const PlacementContext = createContext(null);
export function PlacementProvider({ children }) {
  const [currentUser, setCurrentUser] = useState(null);
  const [applications, setApplications] = useState([]);
```

```
const [drives, setDrives] = useState([]);

const [stats, setStats] = useState(null);

const [loading, setLoading] = useState(true);

const fetchState = useCallback(async () => {

  try {

    setLoading(true);

    const [userRes, appsRes, drivesRes, statsRes] = await Promise.all([

      api.get('/auth/me').catch(() => ({ data: null })),

      api.get('/applications').catch(() => ({ data: [] })),

      api.get('/drives').catch(() => ({ data: [] })),

      api.get('/applications/stats').catch(() => ({ data: null }))

    ]);

    if (userRes.data) setCurrentUser(userRes.data);

    if (appsRes.data) {

      setApplications(appsRes.data);

      localStorage.setItem('cached_applications', JSON.stringify(appsRes.data));

    }

    if (drivesRes.data) {
```

```
        setDrives(drivesRes.data);
        localStorage.setItem('cached_drives', JSON.stringify(drivesRes.data));
      }
      if (statsRes.data) setStats(statsRes.data);
    } catch (err) {
      console.warn('Network unavailable, falling back to local cache', err);
      const cachedApps = localStorage.getItem('cached_applications');
      const cachedDrives = localStorage.getItem('cached_drives');
      if (cachedApps) setApplications(JSON.parse(cachedApps));
      if (cachedDrives) setDrives(JSON.parse(cachedDrives));
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    fetchState();
  }, [fetchState]);

  const updateApplicationStage = async (applicationId, stage, offerStatus,
packageAmount) => {
```

```
  const res = await api.patch(`/applications/${applicationId}/status`, {
    stage,
    offerStatus,
    package: packageAmount
  });
  // Instant update
  await fetchState();
  return res.data;
};
return (
  <PlacementContext.Provider value={{
    currentUser,
    applications,
    drives,
    stats,
    loading,
    refreshData: fetchState,
    updateApplicationStage
  }}>
```

```
      {children}

    </PlacementContext.Provider>

  );

}

export const usePlacement = () => useContext(PlacementContext);
```

5. Prompt to Give Antigravity

Copy and paste the prompt below directly to Antigravity to execute the refactoring:Please refactor and modernize the placement-tracker project in the existing folder using the following plan:

1. Backend Database Setup:

- In backend/prisma/schema.prisma: Create models for User, Company, PlacementDrive, Application, and AuditLog as defined in the Implementation Guide.

- Run npx prisma migrate dev --name init and execute backend/prisma/seed.js to load the ~100 placement application records.

- In backend/src/: Implement authenticateToken and requireRole middleware, wire up JWT login/session endpoints, and add REST routes for drives, applications, and audit logs.

2. AI Assistant Fix:

- In frontend/src/lib/assistantMatcher.js: Implement the fuzzy matching intent parser using Fuse.js and text normalization.

- Ensure that queries like "Where did I apply?", "what is my package?", and "am i shortlisted?" reliably match their corresponding intents and never trigger fallback error messages.

- In frontend/src/components/AIAssistantModal.jsx: Connect the matcher directly to the authenticated student's scoped applications.

3. State & UI Synchronization:

- In frontend/src/services/api.js: Set up the Axios instance with credentials support.

- In frontend/src/context/PlacementContext.jsx: Implement the global context provider with offline localStorage fallback.

- Wrap frontend/src/App.jsx with <PlacementProvider>.

- Update StudentDashboard.jsx, OfficerDashboard.jsx, and AdminDashboard.jsx to consume live data and trigger updateApplicationStage() with immediate reflection.

4. Verify that logging in with admin@college.edu, officer@college.edu, and riya.sharma@college.edu correctly loads persistent records from the database and that updating stages in the Officer portal immediately updates the student metrics.